

1. 引言 (Introduction)

这部分主要介绍文档本身，而不是软件细节。

- **1.1 目的 (Purpose)**
 - 说明编写这份文档是为了什么（例如：确立开发基准、作为验收依据）。
 - 指明文档的预期读者（开发人员、测试人员、项目经理、用户代表等）。
- **1.2 文档范围 (Scope)**
 - 给待开发的软件起个名字。
 - 简要说明软件做什么，不做什么（In scope vs Out of scope）。
- **1.3 定义、缩写词和缩略语 (Definitions, Acronyms, and Abbreviations)**
 - 解释文档中出现的专业术语（如：API, SKU, PV/UV）。
- **1.4 参考文献 (References)**
 - 列出相关的文档（如：合同、上级需求文档、UI设计稿链接、遵循的行业标准）。
- **1.5 文档概览 (Overview)**
 - 简单介绍文档后续章节的安排。

2. 总体描述 (Overall Description)

这部分是从宏观角度（业务视角）描述系统，不涉及具体技术细节。

- **2.1 产品前景 (Product Perspective)**
 - 说明该软件与外部系统的关系（是独立产品？还是大系统中的个子模块？）。
 - 画出系统上下文图（Context Diagram），展示系统与外部实体（人、其他系统）的交互。
- **2.2 产品功能概要 (Product Functions)**
 - 列出系统的主要功能列表（High-level features），可以用列表或用例图表示。
- **2.3 用户特征 (User Characteristics)**
 - 描述谁会使用系统（如：管理员、普通访客、VIP用户）。
 - 说明他们的技术水平、受教育程度等，这影响UI设计的复杂度和帮助文档的编写。
- **2.4 约束 (Constraints)**
 - 列出限制开发自由度的因素（如：必须用Java开发、必须在3个月内交付、预算限制、必须兼容IE浏览器等）。
- **2.5 假设和依赖 (Assumptions and Dependencies)**
 - 假设：假设网络环境是稳定的、假设第三方API是可用的。
 - 依赖：项目依赖某个尚未发布的外部库，或者依赖硬件设备的到位时间。

3. 具体需求 (Specific Requirements) —— 【核心部分】

这是文档最长、最重要的部分，所有开发和测试都基于此。通常有两种组织方式：按功能模块划分或按用户角色划分。以下是按类型划分的标准结构：

- **3.1 外部接口需求 (External Interface Requirements)**
 - **3.1.1 用户界面 (User Interfaces)**：对应UI原型，说明屏幕布局、按钮行为、快捷键等。
 - **3.1.2 硬件接口 (Hardware Interfaces)**：系统与读卡器、打印机、传感器等的交互协议。
 - **3.1.3 软件接口 (Software Interfaces)**：与数据库、操作系统、第三方支付API的接口定义。
 - **3.1.4 通信接口 (Communication Interfaces)**：HTTP/HTTPS、TCP/IP、FTP等协议标准。
- **3.2 功能需求 (Functional Requirements)**

- 这是详细的逻辑描述, 通常针对每一个功能点 (Feature) :
- **3.2.1 功能名称/标识符**
 - **描述:** 简述功能作用。
 - **输入:** 输入的数据类型、格式、校验规则。
 - **处理逻辑:** 系统如何处理数据 (算法、逻辑判断、数据库操作) 。
 - **输出:** 返回的结果、错误提示、跳转页面。
- **3.2.2 [下一个功能]...**
- **3.3 性能需求 (Performance Requirements)**
 - **静态数值:** 支持多少并发用户? 数据库最大容量?
 - **动态数值:** 页面加载时间<2秒? API响应时间<500ms?
 - **吞吐量:** 每秒处理多少事务 (TPS)?
- **3.4 逻辑数据库需求 (Logical Database Requirements)**
 - 数据实体及其关系 (ER图) 。
 - 数据保留策略 (日志保留半年等) 。
- **3.5 设计约束 (Design Constraints)**
 - 遵循的标准 (如 ISO 9001) 。
 - 代码规范 (如 Google Java Style) 。
- **3.6 软件系统属性 (Software System Attributes)**
 - **可靠性 (Reliability):** 平均无故障时间 (MTBF)。
 - **可用性 (Availability):** 7x24小时运行, 年宕机时间不超过XX分钟。
 - **安全性 (Security):** 权限控制、加密传输、防SQL注入、数据备份。
 - **可维护性 (Maintainability):** 模块化程度、注释要求。
 - **可移植性 (Portability):** 是否需要跨平台 (Windows/Linux/Mac) 。

4. 附录 (Appendices)

- **4.1 待定问题 (TBD List)**
 - 当前尚未确定、需要后续沟通确认的需求。
- **4.2 原始分析模型**
 - 复杂的算法说明、额外的流程图、旧系统的报表样本等。

💡 编写小贴士

1. **编号管理:** 给每一个需求一个唯一的ID (如 **REQ-USER-001**) 。这样在后续的用例 (Test Case) 中可以引用该ID, 实现需求的**可追溯性 (Traceability)**。
2. **避免歧义:**
 - ☒ **正确:** 系统应在用户点击“保存”后 1秒内 弹出成功提示。
 - ☒ **错误:** 系统应 快速 响应用户的操作。
3. **敏捷与瀑布的区别:**
 - 如果是**瀑布模型**, 上述文档需要写得非常详尽, 几十甚至上百页。

- 如果是**敏捷开发**，SRS可能会被简化，核心的“3.2 功能需求”通常被拆分为**用户故事 (User Stories)** 放在管理工具（如Jira）中，文档只保留架构和非功能需求部分。